

ONLINE MASTERS IN **DATA SCIENCE**

DSC258: NATURAL LANGUAGE PROCESSING

NAME ENTITY RECOGNITION

UC San Diego

COMPUTER SCIENCE & ENGINEERING
HALICIOĞLU DATA SCIENCE INSTITUTE

Classic Named Entity Recognition (NER)

Some questions to think about:

- Can massive raw texts help?
- Can dictionaries help?
- Are human annotations always correct?
- Is Tokenizer always good?

Wikipedia:

- **Named-entity recognition (NER)** is a subtask of **information extraction (IE)** that seeks to **locate** and **classify named entities** in text into **pre-defined categories**.
- In IE, A **named entity** is a real-world object.

Example

- Input:
 - Jim bought 300 shares of Acme Corp. in 2006.
- Output:
 - [Jim]_{Person} bought 300 shares of [Acme Corp.]_{Organization} in [2006]_{Time}.

Sequence labeling framework

Two popular schemes

- BIO: **B**egin, **I**n, **O**ut
- BIOES: **B**egin, **I**n, **O**ut, **E**nd, **S**ingleton
- BIOES is arguably better than BIO (Ratinov and Roth, ACL 09)

Example:

- LABELS: [Jim]_{Person} bought 300 shares of [Acme Corp.]_{Organization} in [2006]_{Time}.
- TOKENS: Jim bought 300 shares of Acme Corp. in 2006 .
- BIO: B-PER 0 0 0 0 B-ORG I-ORG 0 B-Time 0
- BIOES: S-PER 0 0 0 0 B-ORG E-ORG 0 S-Time 0

Handcrafted Features + Conditional Random Fields

Popular Tools

- Stanford NER (Finkel et al., ACL'05)
- <https://nlp.stanford.edu/software/CRF-NER.shtml>
- Ohio State University Twitter NER (Ritter et al., EMNLP'11, KDD'12)
- https://github.com/aritter/twitter_nlp

Bag-of-words Features

- Unigrams, Bigrams, ...
- Lowercase
- Lemmization
- Stemming
- Part-of-speech Tags
- Word Shape

Context Features

- E.g., Left/right 4 words

Character n-grams

- Useful for unknown words

Brown Clusters

- Data-driven similar words
- Alleviate sparsity issues

WordNet Clusters

- Semantically similar words

Dictionaries

- Domain-specific dictionaries

Conditional Random Field (CRF) model

- Capture the dependencies between labels

Words and Characters

(Auto Extracted) Features

$$p(\hat{\mathbf{y}} | \mathbf{x}_i, \mathbf{c}_i) = \frac{\prod_{j=1}^n \phi(\hat{y}_{j-1}, \hat{y}_j, \mathbf{z}_j)}{\sum_{\mathbf{y}' \in \mathbf{Y}(\mathbf{z})} \prod_{j=1}^n \phi(y'_{j-1}, y'_j, \mathbf{z}_j)}$$

Predicted label sequence

All valid label sequences

They require:

- Human Expert Efforts = Domain + Linguistic
- Every single sentence must be fully annotated

Therefore, it is difficult to adapt them to new domain from the original training domain

- New training data → Expensive in specific domains (e.g., biomedical)

Two pioneer models

- LSTM-CRF (Lample et al., NAACL'16)
- LSTM-CNN-CRF (Ma and Hovy, ACL'16)

	LSTM-CRF	LSTM-CNN-CRF
Word-Level	Bidirectional LSTMs	Bidirection LSTMs
Character-Level	Bidirectional LSTMs	Convolutional NN

- The first neural model that outperforms the models based on handcrafted features

LET'S TRY SPACY FOR NER TOGETHER

```
In [1]: import spacy
```

```
nlp = spacy.load('en_core_web_sm')
```

```
In [2]: sentence = "Apple is looking at buying U.K. startup for $1 billion"
```

```
doc = nlp(sentence)
```

```
for ent in doc.ents:
```

```
    print(ent.text, ent.start_char, ent.end_char, ent.label_)
```

```
    print('\t', sentence[ent.start_char : ent.end_char]) # the start_char and end_char are offs
```

```
Apple 0 5 ORG
```

```
    Apple
```

```
U.K. 27 31 GPE
```

```
    U.K.
```

```
$1 billion 44 54 MONEY
```

```
    $1 billion
```

Capitalization

Some old version of spaCy is sensitive to capitalization but the new version now is using deep neural network models, which is much better.

spaCy v2.0's Named Entity Recognition system features a sophisticated word embedding strategy using subword features and "Bloom" embeddings, a deep convolutional neural network with residual connections, and a novel transition-based approach to named entity parsing.

```
In [3]: sentence = "apple is looking at buying U.K. startup for $1 billion"

doc = nlp(sentence)

for ent in doc.ents:
    print(ent.text, ent.start_char, ent.end_char, ent.label_)
```

```
U.K. 27 31 GPE
$1 billion 44 54 MONEY
```